Bitcamp | Quantum Track

eleQt: Quantum Oblivious Transfer with Ekert-91

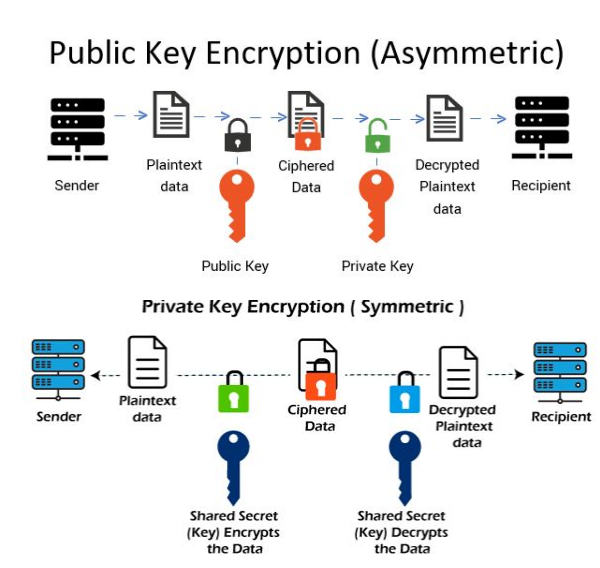
Team: Untitled
Pratham Gujar, Prakriti Shahi, Jonathan Yang

Objectives

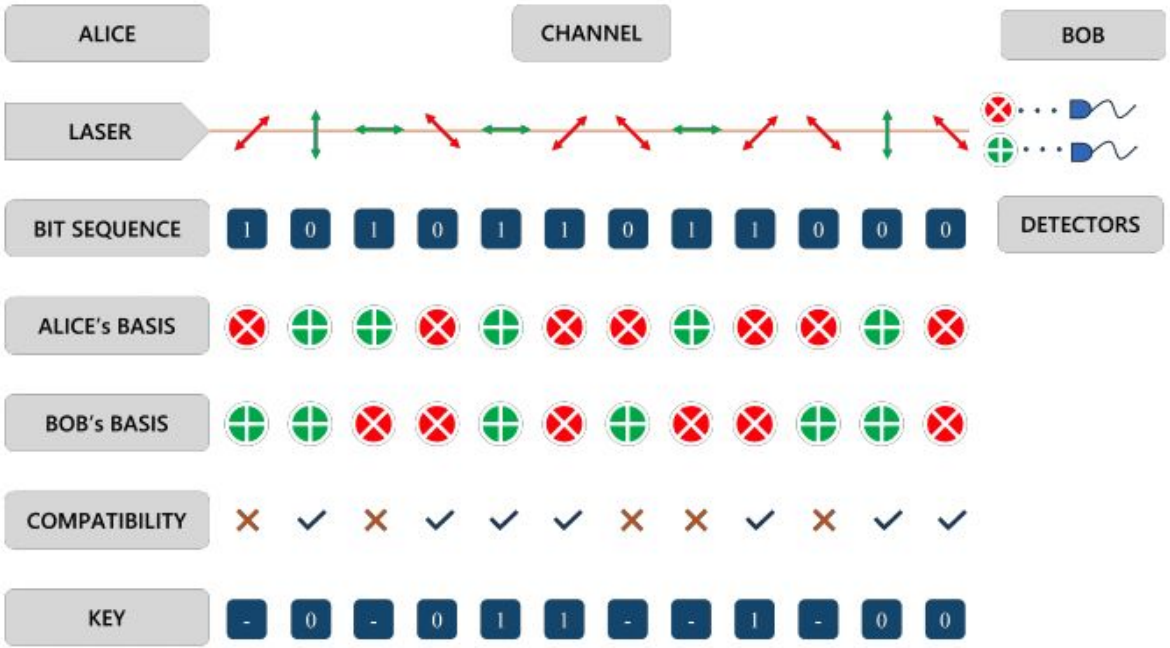
- To demonstrate Quantum Oblivious Transfer (a secure multiparty computation protocol) in a practical real-life scenario
- Perform the quantum cryptography step using quantum teleportation instead of regular quantum communication
- Quantum Voting Machines:
    - No voter or third party should be able to see any individual voter's vote
    - Sum of votes to each party should be computed such that every step of the computation is secure
- Build a universal (classical) gate set that is quantum-secure so that *any* computation is feasible

Classical Cryptography and the Need for Quantum Cryptography

- Classical cryptography uses computationally complex one-way functions to encrypt information
- RSA cryptography works using the prime-factorization problem with **public key cryptography** - computationally difficult to prime factorize a large number
- Shor's algorithm can break RSA cryptography
- Solution: **Private key cryptography**
- But how do we share the private key?
- Enter → **Quantum Key Distribution**
- Earliest protocol: BB84

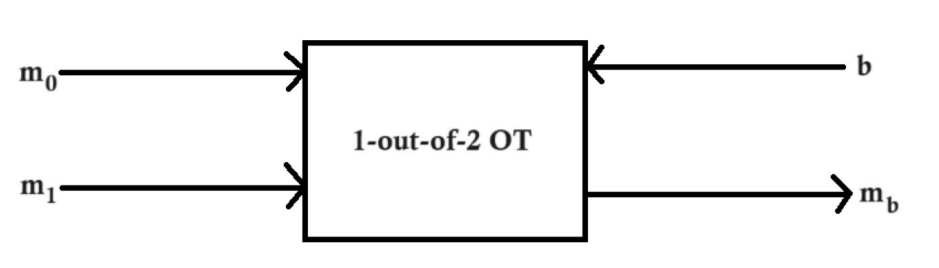


BB84 Protocol

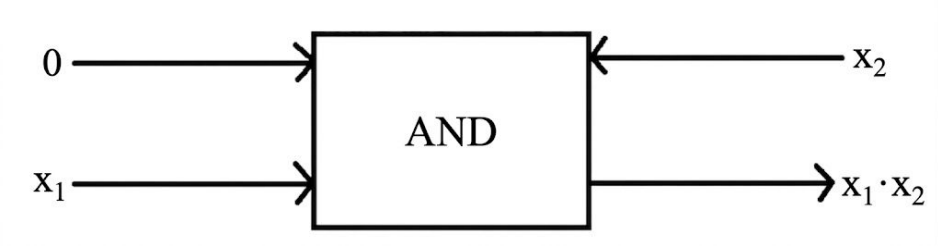


Oblivious Transfer

- Secure multi-party computation



- AND using 1-out-of-2 OT:



- NOT is unary ⇒ inherently secure
- NOT + AND create a universal gate set
- ∴ Secure AND + NOT = secure arbitrary computation
- QOT just requires a quantum implementation of 1-out-of-2 OT

Quantum Oblivious Transfer: BBCS Protocol

Parameters: Security parameter, n; two-universal family of hash functions, F.
Alice (A) input: $(m_0, m_1) \in \{0,1\}^l$ (two messages).
Bob (B) input: $b \in \{0,1\}$ (bit choice).

BB84 phase:

1. Alice generates random bits $x^A \in \{0,1\}^n$ and random bases $\theta^A \in \{+,\times\}^3$. Alice sends the state $\left|x^A\right\rangle_{\theta^A}$ to Bob.
2. Bob randomly chooses bases $\theta^B \in \{+,\times\}$ to measure the received qubits. We denote his output bit string as x^B .

Oblivious key phase:

3. Alice reveals to Bob the bases θ^A used during the BB84 phase and sets her oblivious key to $ok^A = x^A$.
4. Bob computes $e^B = \theta^A \oplus \theta^B$ and sets $ok^B = x^B$.

Transfer phase:

5. Bob defines $I_0 = \{i : e_i^R = 0\}$ and $I_1 = \{i : e_i^R = 1\}$ and sends the state I_b to Alice.
6. Alice picks two uniformly random hash functions $f_0, f_1 \in F$, computes the pairs of strings (s_0, s_1) as $s_i = m_i \oplus (ok^A_{I_{b\oplus i}})$ and sends the pairs (f_0, f_1) and (s_0, s_1) to Bob.
7. Bob computes $m_b = s_b \oplus f_i(ok^B_{I_0})$.

BBCS in the F_com-hybrid model

Parameters: Security parameter, n; two-universal family of hash functions, F.
Alice (A) input: $(m_0, m_1) \in \{0,1\}^l$ (two messages).
Bob (B) input: $b \in \{0,1\}$ (bit choice).

BB84 phase: same as BBCS

Cut and choose phase:

3. Bob commits to the bases and measured qubits, i.e., $\mathbf{com}(\theta^B, x^B)$, and sends this to Alice. For an example of such a commitment scheme, refer to [12].
4. Alice asks Bob to open a subset T of commitments (e.g. $\frac{n}{2}$ elements) and receives $\{\theta_i^B, x_i^B\}_{i \in T}$.
5. In case any opening is not correct, i.e., $x_i^B \neq x_i^A$ for $\theta_i^B = \theta_i^A$, abort. Otherwise, proceed to the oblivious key phase.

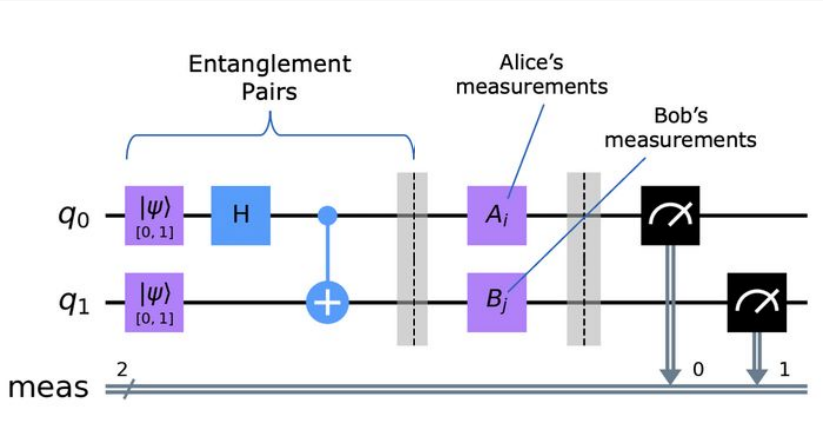
Oblivious key phase: same as BBCS

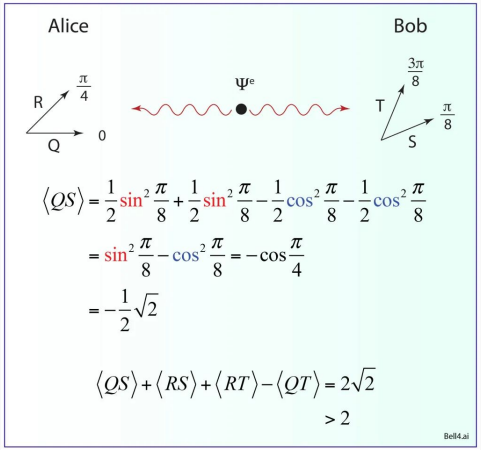
Transfer phase: same as BBCS

Security: The BBCS protocol in the F_{com}-hybrid model is secure against a dishonest sender as well as a dishonest receiver.

Replacing BB84 with Ekert-91

- Instead of Alice sending a qubit-string to Bob, both Alice and Bob receive an entangled (Bell) pair of qubits
- Detection of evesdropper (Eve) during BB84 is based on No-Cloning theorem; but there is still a non-zero chance that Eve is not detected (scales with 1/(2^n))
- In E-91, Eve destroys entanglement between the qubits which can be detected with 100% certainty using Bell or CHSH inequalities





QSVP: Quantum Secure Voting Protocol

- Original protocol/example
- Quantum voting protocol so all voters' votes are completely hidden
- Binary voting: 2 parties, represented by 0 and 1 respectively
    - If total number of voters known (easy - counter), sum of all votes is number of votes to party B and total - B = no. of votes to party A
    - Implemented using ripple half-adders involving AND, NOT, and XOR gates
    - XOR is free/secure due to secret sharing, but every AND requires 2 OT protocols
- Multi-party voting: Cannot just sum directly
    - Each vote represented by an m-bit string (for m parties)
    - Each *valid* vote has a single 1 in the m-bit string (Zero Knowledge Proof)
    - Counter (implemented using ANDs (OTs), NOTs, and XORs) for vote-counting

Code Complexity

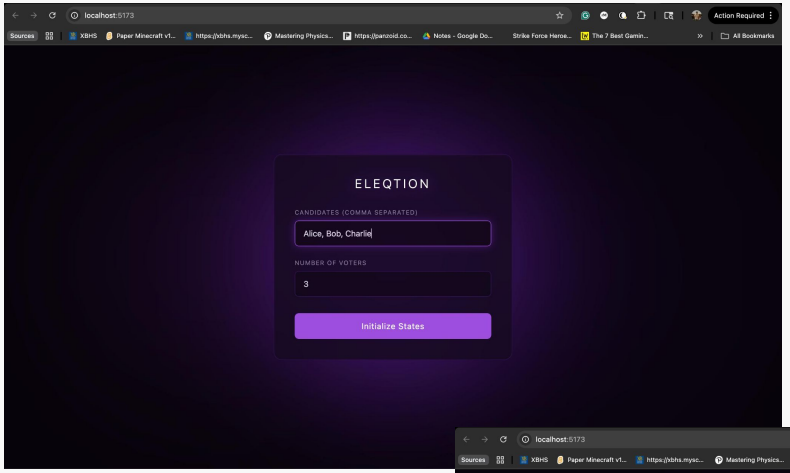
For m = number of parties and n = number of voters,

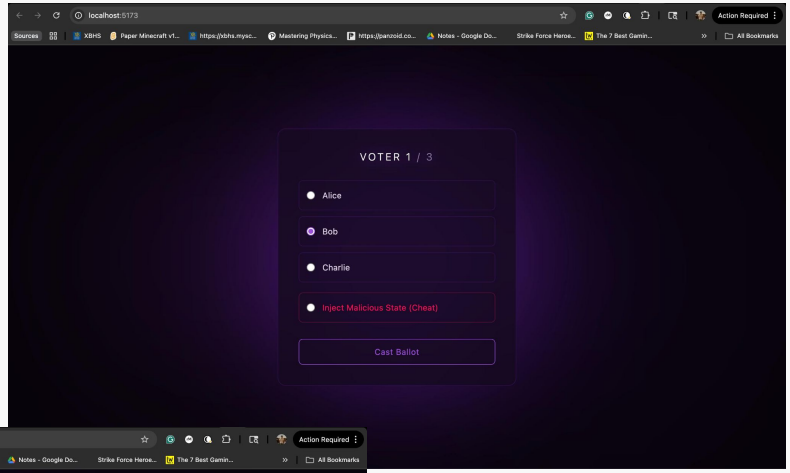
- No. of qubits ~O(m•n•logn)
- Total classical work (time) ~ O(m•n•logn)
- Total quantum circuit depth ~O(m•n•logn)

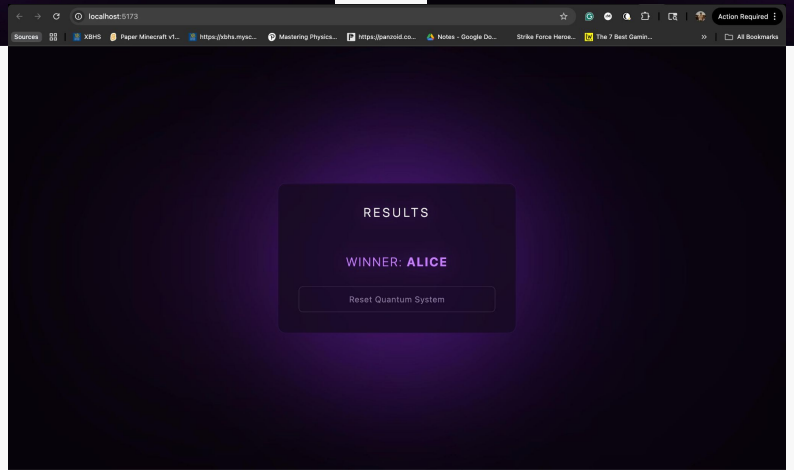
m will usually be a small number, so effectively, all things scale ~O(n•logn) which is close to optimal (natural result of the ripple carry adders)

Implemented code using both simulator and actual quantum hardware - IBM Kingston and IBM Fez!

Interactive GUI (website demo)







Github Deliverables

eleqt_binary_hardware.ipynb

- 2-party voting implemented on IBM quantum hardware

eleqt_multiparty.ipynb

- Multiparty voting implemented with qiskit simulators

eleqtion-site (GUI app)

- node_modules (folder): npm packages (axios, tailwindcss)
- public (folder): pics and icons
- src (folder): contains main code
  App.jsx, index.css, etc.
- Other necessary folders and files

README.md and complexity_analysis.md

Code Demo: https://github.com/prakriti-shahi/bitcamp

Thank You!